

Algoritmi e Diagrammi a blocchi

Algoritmo

Prima di vedere i diagrammi a blocchi bisogna capire cosa si intende per algoritmo:

Un algoritmo è una descrizione di un percorso di risoluzione di problemi (problem solving) per raggiungere i risultati finali partendo da dati iniziali.

Scriviamo l'algoritmo come se ci stessimo rivolgendo a un esecutore, in grado di eseguire azioni descritte come istruzioni, scritte in un linguaggio specifico.

Supponiamo che l'esecutore sia a disposizione di un utente (non necessariamente la persona che ha scritto l'algoritmo) che utilizza l'esecuzione dell'algoritmo.

Istruzioni	Finalità
Input	Riceve dati (numeri, espressioni, testi)
Output	manda messaggi e comunica i risultati
Assignment	immagazzina un pezzo di dato associandolo con un nome variabile
Calculation	esegue le operazioni di un dato

Ci permettono di rappresentare degli algoritmi in maniera visiva

Pseudo-codice

Pseudo-codice: traduzione in linguaggio del codice e viene usata per abbozzare gli algoritmi prima di scriverli.

Struttura fondamentale

Fundamental Structures

Sequence

```
Start
<instruction>
<instruction>
<instruction>
...
End
```

Selection

```
IF <condition>
  THEN
    <instructions1>
  ELSE
    <instructions2>

or

IF <condition>
  THEN
    <instructions1>
```

Iteration

```
REPEAT
  <instructions>
UNTIL <condition>

or

WHILE <condition> DO
  <instructions>
END WHILE

or

FOR <index> FROM <start> TO <end>
  DO<instructions>
NEXT <index>
```



Struttura fondamentale.png

Diagramma a blocchi

Questo tipo di diagramma lo si scrive seguendo un ordine e una struttura precisa.

Opening of the algorithm	Closing of the algorithm	Reading input data (input)	Communicating messages and/or results (output)	Assigning data and/or performing calculations	Controlling the truth value of a condition
Start	End	Read	Write	Assign	Condition

Struttura diagramma a blocchi.png

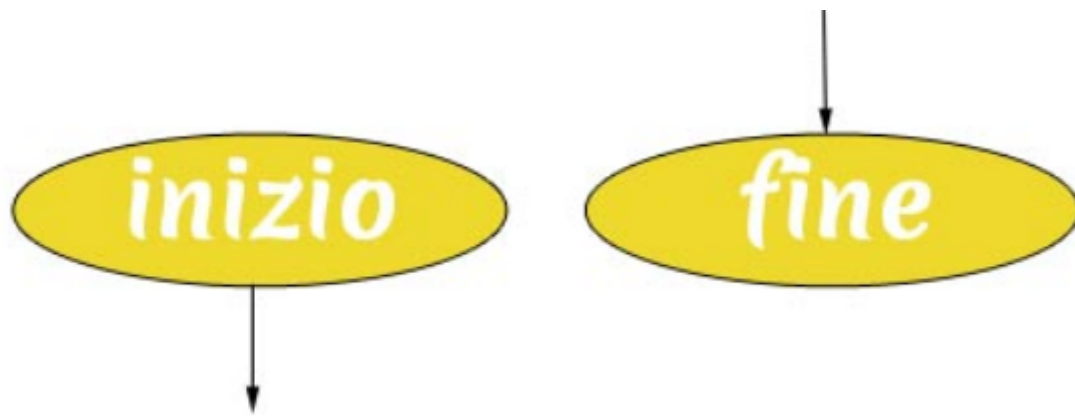
Un diagramma a blocchi è un modo visivo di rappresentare il flusso di un programma. Esso mostra i vari passaggi (o istruzioni) e come vengono eseguiti in sequenza, selezione o iterazione.

Struttura

1. Elisse

È utilizzata per indicare l'inizio o la fine di un diagramma a blocchi.

Quindi ciascun diagramma inizierà con il blocco **inizio** e terminerà, dopo aver risolto il compito assegnato, con il blocco **fine**



Blocchi di inizio_fine.png

2. Parallelogramma

È utilizzato per prendere dei dati in input o per visualizzare dei dati in output.

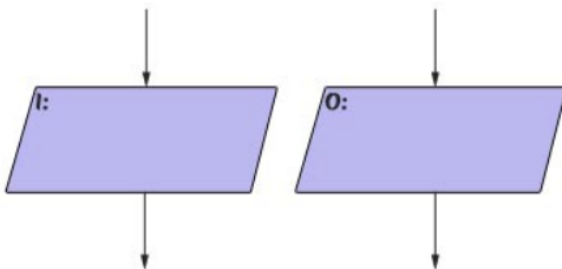
La nomenclatura prevede di scrivere per l'input **"Read"** (scrivere accanto la variabile da mettere in input) e per l'output **"Write"** (scrivere accanto l'output che si vuole visualizzare), oppure è consigliabile inserire una **I** in alto a sinistra, seguita dai due punti. Similmente per l'output, che si è soliti indicare con una **O** in alto a sinistra, sempre seguita dai due punti.

Primo esempio di I/O:



I_O.png

Secondo esempio di I/O:

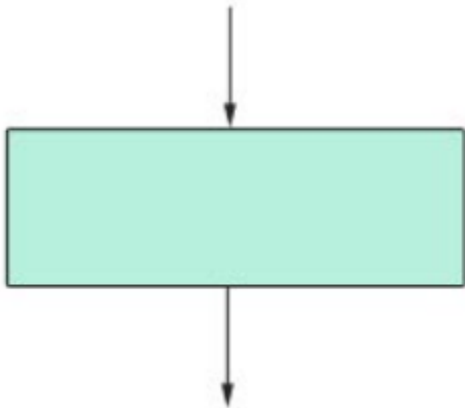


I_O2.png

3. Rettangolo:

Il rettangolo è utilizzato per eseguire dei calcoli, ovvero per elaborare dei dati.

Ad esempio per calcolare la somma tra due numeri o l'area di un rettangolo o ancora la media fra tre numeri, ecc...



Rettangolo Block Diagram.png

4. Rombo:

Il rombo è utilizzato per le istruzioni condizionali, ovvero per porre una domanda.

All'interno dunque viene fatto un **test**, per cui si valuta una condizione che può essere o vera o falsa, quindi si sceglie tra due strade diverse. Un esempio di semplice test potrebbe essere quello di vedere se un numero è positivo o negativo



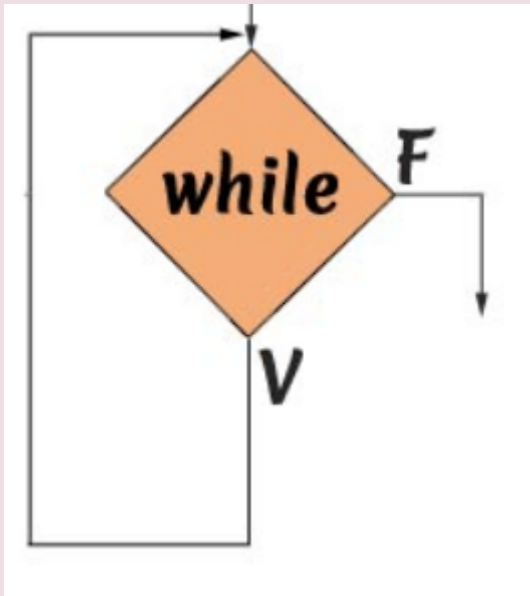
Rombo Block Diagram.png

Come vedremo più avanti, il rombo viene spesso utilizzato per i cicli `if`, `elif`, `else`, `while` e `do-while`.

🔗 Cosa sono i cicli >

I **cicli** sono delle **strutture iterative** che hanno lo scopo di ripetere più volte una o più istruzioni contenute al loro interno.

Example



Cicli while Block Diagram.png

Esempio di un Diagramma a blocchi

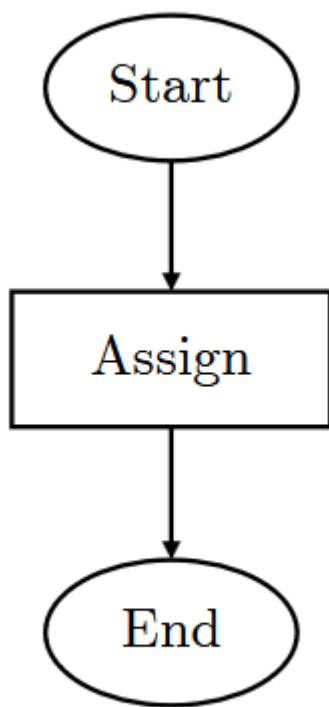
Ecco come potremmo strutturare un diagramma a blocchi che somma i numeri in una lista:

1. Sequenza

La sequenza è rappresentata dai blocchi che si susseguono uno dopo l'altro .

In questo caso, la sequenza inizierà con il blocco che definisce la lista e continuerà con un blocco che inizializza una variabile per la somma.

I blocchi di apertura e di chiusura degli algoritmi hanno una sola freccia, che, rispettivamente entra ed esce, mentre i blocchi di mezzo hanno 2 frecce: una in entrata e una in uscita.



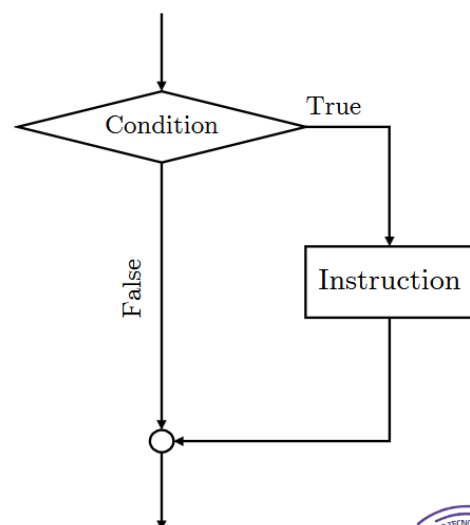
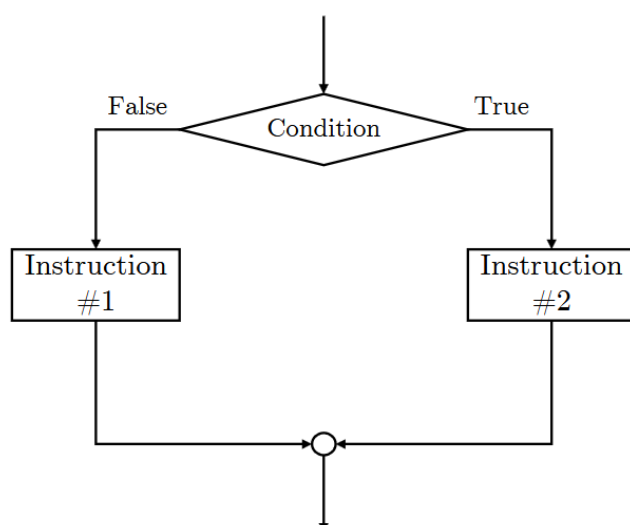
Block sequence.png

2. Selezione (Condizione)

La selezione permette di eseguire un blocco di codice solo se una condizione è vera.

Non è strettamente necessaria per la somma, ma può essere usata per controllare se la lista è vuota o se ha numeri da sommare.

Selection Block



Analizzando l'immagine, partendo da sinistra:

1. Primo Diagramma:

- Condizione:
 - La decisione si basa su una condizione logica. La forma a rombo rappresenta questa decisione.
- Se la condizione è **vera (True)**, il flusso si sposta verso **Instruction #2**.
- Se la condizione è **falsa (False)**, il flusso si sposta verso **Instruction #1**.
- Istruzione eseguite:
 - **Instruction #1** viene eseguita se la condizione è falsa.
 - **Instruction #2** viene eseguita se la condizione è vera.
- Dopo aver eseguito l'istruzione (1 o 2), il flusso torna al punto di uscita comune.

≡ Example

Questo diagramma in parole povere rappresenta una selezione `if-else`, cioè:

```
if condition:
    instruction_2()
else:
    instruction_1()
```

2. Secondo diagramma:

- Condizione:
 - Come nel diagramma precedente, il flusso dipende da una condizione.
- Se la condizione è **vera (True)**, viene eseguita l'**Instruction**.
- Se la condizione è **falsa (False)**, il flusso salta direttamente al punto di uscita senza eseguire nulla.

≡ Example

Questo modello rappresenta una selezione semplice, dove un'azione viene eseguita solo se una condizione è soddisfatta.

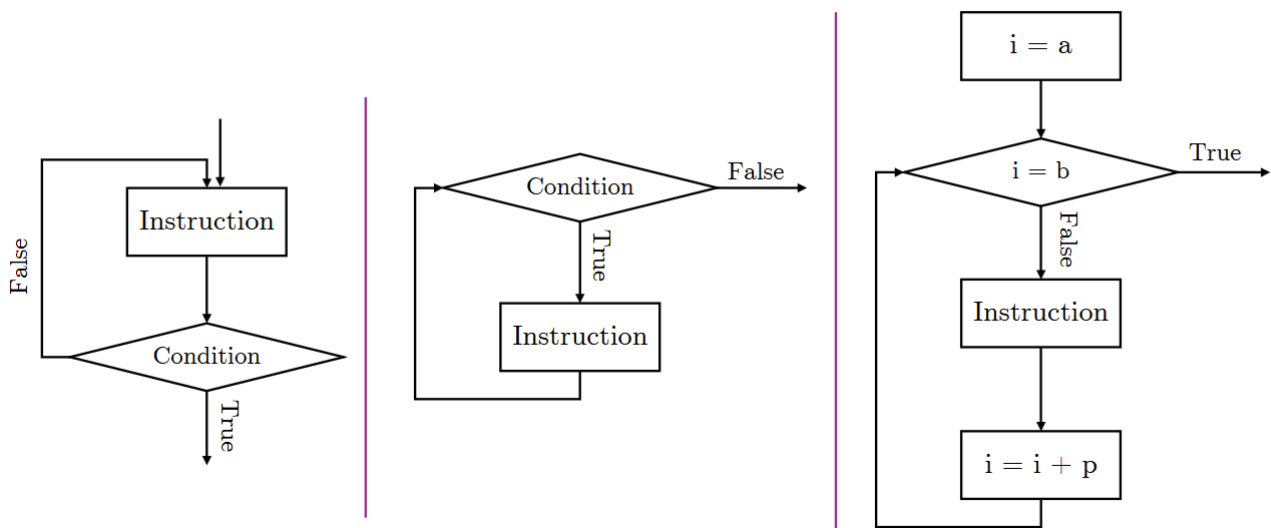
```
if condition:  
    instruction()
```

3. Iterazione

L'iterazione è il processo che permette di ripetere una serie di operazioni più volte.

Per sommare i numeri in una lista, si utilizza un ciclo (iterazione). In un diagramma a blocchi, questo ciclo è rappresentato da un blocco "loop" (per esempio, "for" o "while") che scorre la lista.

Control Block



Control block.png

Analizzando l'immagine, partendo da sinistra verso destra:

1. Primo Diagramma:

Descrizione:

Rappresenta un ciclo condizionale, simile a un ciclo `while`, in cui una condizione viene valutata

Funzionamento:

- La condizione viene verificata
- Se la condizione è vera (`true`), il blocco di istruzione viene eseguito
- -Dopo aver eseguito le istruzioni, il flusso ritorna al controllo della condizione.
- Se la condizione è **falsa (False)**, il ciclo termina, e il programma prosegue verso l'uscita del ciclo.

≡ Esempio Pratico

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Qui il ciclo continua a stampare e incrementare il valore di `i` finché `i` è minore di 5.

2. Secondo Diagramma:

Descrizione:

Questo diagramma rappresenta un altro tipo di ciclo condizionale, molto simile al primo. Anche qui si verifica una condizione e, se è **vera**, il programma esegue un'istruzione e ritorna al controllo della condizione.

- Il flusso si sposta in modo più evidente dalla condizione all'istruzione. Questo può indicare una **variante del controllo condizionale**, ad esempio un ciclo `do-while` in alcune lingue di programmazione, dove l'istruzione viene eseguita almeno una volta prima di verificare la condizione.

≡ Esempio Pratico:

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Sebbene Sebbene simile al primo, il concetto di esecuzione prima del controllo può essere esplicitato meglio con cicli come `do-while`.

3. Terzo Diagramma:

Descrizione: Questo diagramma rappresenta un ciclo con inizializzazione, controllo e aggiornamento, che corrisponde a un **ciclo for** o a un ciclo strutturato con contatore.

Funzionamento:

- L'inizializzazione viene eseguita (`i = a`).
- Si verifica una condizione (`i = b`).
 - Se è **vera (True)**, il flusso entra nel blocco delle istruzioni.
 - Se è **falsa (False)**, il ciclo termina.
- Dopo aver eseguito le istruzioni, il valore di `i` viene aggiornato (`i = i + p`), e il ciclo ritorna al controllo della condizione.

≡ Esempio pratico

```
for i in range(0, 10, 2):  
    print(i)
```

Quindi:

- `i = 0` è l'inizializzazione.
- `i < 10` è la condizione.
- `i = i + 2` è l'incremento a ogni iterazione.

≡ Ricapitolando

- **Primo diagramma:**
Ideale per cicli basati su condizioni (`while`).
- **Secondo diagramma:**
Potrebbe rappresentare un `do-while` (istruzioni eseguite almeno una volta).
- **Terzo diagramma:**
Rappresenta cicli strutturati con contatore (`for`), dove inizializzazione, condizione e aggiornamento sono chiaramente separati.

Diagramma a blocchi per sommare 5 numeri in una lista:

1. Inizio
2. Inizializza la somma a 0
3. Prendi il primo numero della lista
4. Aggiungi il numero alla somma
5. Verifica se ci sono altri numeri nella lista

- Se ci sono, torna al passo 3.
- Se non ci sono, vai al passo successivo.

6. Stampa la somma totale

7. Fine

Traduzione in pseudocodice:

```
somma = 0
for numero in lista:
    somma += numero
print(somma)
```

In un diagramma a blocchi, il flusso di esecuzione partirebbe da un blocco di inizio, per passare all'inizializzazione della somma (un blocco di assegnazione), poi a un blocco di iterazione "for" che scorre la lista. Dopo ogni iterazione, un blocco di somma aggiungerebbe il valore corrente della lista alla variabile `somma`. Alla fine, quando non ci sono più numeri nella lista, un blocco di stampa restituirebbe il risultato finale.

Rappresentazione visiva:

- Un **blocco rettangolare** (operazione) per l'inizializzazione della somma.
- Un **rombo** (condizione) per verificare se ci sono numeri nella lista.
- Un **blocco parallelogramma** (input/output) per stampare il risultato.
- Un **blocco ciclico** (loop) per il "for".

compito per casa

un diagramma a blocchi che fa la somma di 5 numeri all'interno della lista